

The Four-Accountability-Questions Test as Standalone Procedure Specification: A Verification Procedure for Path Retracement in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one operational test — the four-accountability-questions test — as a standalone verification procedure that determines whether a CKS substrate's provenance content can in fact answer the four accountability questions the source paper's path retraceability commitment is designed to make answerable.

Abstract

The CKS pattern's path retraceability commitment (§3.1, §5) names four accountability questions that any substrate change must allow a reader to answer from substrate content alone: who made the change, when it was made, under what authority or rule, and through what mechanism. A separate derivation note in this series formalizes the architectural mechanism that supports answerability — six provenance fields substrate content must carry — and a sibling operational test verifies that those fields are present. This note formalizes the complementary operational test: it verifies that, given fields present, the four questions are in fact answerable from substrate provenance for any substrate change, with answers that are specific (not aggregated to a window), correct (not generic placeholders), and properly attributed (distinguishing humans authoring rules from AI mediating consultations under those rules). The test pairs with the provenance-completeness test to operationally verify path retraceability in two halves: fields present (architectural mechanism) and questions answerable (operational outcome). The note states what the test verifies, specifies the procedure across substrate change categories, names pass and fail criteria, enumerates the anti-patterns the test specifically detects, locates the test within deployment verification, and states its limits.

1. Why the four-accountability-questions test needs to be formalized as standalone

The path retraceability commitment is what allows a reader, given the substrate as it stands, to answer "what produced this content, by whom, under what authority, and through what mechanism?" by reading substrate content alone. The commitment is not satisfied by recording events somewhere; it is satisfied by recording the right state — provenance attached to the content the reader is reading — in the substrate the reader is reading. A separate derivation note formalizes the architectural mechanism:

six provenance fields (writer attribution, timestamp, antecedent reference, rule reference, mechanism identifier, rationale where required) that substrate content must carry to make the path reconstructable. A sibling operational test verifies that those fields are present.

Field presence is necessary but not sufficient. The deployment-relevant question is not whether the fields exist as schema slots but whether the four accountability questions §5 of the source paper names — Q1 *who*, Q2 *when*, Q3 *under what authority or rule*, Q4 *through what mechanism* — are in fact answerable from those fields, for any substrate change, with answers that withstand the operational requirements the commitment carries. A substrate may have all six fields populated and still fail to answer the questions, because field presence does not guarantee answer specificity, answer correctness, or proper attribution distinction between humans authoring rules and AI mediating consultations under those rules. This note formalizes the test that distinguishes architectural mechanism from operational outcome; together with the provenance-completeness sibling test, it verifies path retraceability operationally — one test for the mechanism, one for the outcome.

2. The architectural commitment under test

The test verifies the four accountability questions §5 of the source paper names as the operational content of what retraceability is for: any substrate change must permit a reader to answer

- **Question 1 — the *who* question.** Who made the change — and specifically, whether the change was a direct human action (a human exercising the modify or override right), a human-authored orchestration rule update, an AI-mediated cell processing event under such a rule, or an orchestration-triggered substrate write. The answer must distinguish the actor categories per the AI-as-substrate-mediator commitment, not collapse them into "the system."
- **Question 2 — the *when* question.** When was the change made — at a specific timestamp recoverable from substrate provenance, not at an aggregated time window or a session-level boundary.
- **Question 3 — the *under what authority* question.** Under what authority or rule was the change authorized — with a specific rule reference resolvable to a specific orchestration rule, not a generic policy designation.
- **Question 4 — the *through what mechanism* question.** Through what mechanism was the change executed — with a specific cell identifier or named mechanism, not an abstract "automated process" or "system action" placeholder.

The four questions are individually addressable in the source paper and individually testable here. The test operationalizes their joint answerability under the conditions §3.3 attaches to retraceability: at any time, from substrate content alone, with answers specific to the change in question.

3. The test procedure

The test procedure is a structured walk over the categories of substrate changes a CKS deployment can produce, with each change's provenance interrogated against the four questions. The procedure has six steps.

Step 1 — Enumerate the change categories under test. The deployment must carry substrate changes from each category the source paper's commitments produce: direct human modifications, human overrides, human-authored orchestration rule updates, AI-mediated cell processing events under orchestration rules, and orchestration-triggered cell or substrate writes. Categories absent from the deployment are recorded as "not applicable" and excluded from the test scope rather than declared passed by default.

Step 2 — Sample substrate changes within each category. From each applicable category, the test selects a sample drawn from substrate content actually accumulated in the deployment, not from synthetic test data — at least one change per category at minimum, more when the category has internal variation (multiple orchestration rules, multiple cell types, multiple AI mediation patterns).

Step 3 — For each sampled change, attempt to answer the four questions from substrate provenance alone. The test reader queries the substrate's provenance content and constructs an answer to each of Q1, Q2, Q3, and Q4. The reader is permitted to use the substrate's own indexing and lookup affordances; the reader is not permitted to consult external logs, the LLM mediator's session history, vendor audit dashboards, or human recollection. The constraint matches the source paper's commitment that retraceability is satisfied by reading substrate content alone (§3.1).

Step 4 — Verify answer specificity. Each answer must be specific to the change in question: a specific actor identifier for Q1, a specific timestamp for Q2, a specific rule identifier for Q3, a specific cell or mechanism identifier for Q4. Aggregated answers ("a session that ran on Tuesday," "policy framework X," "the standard pipeline") fail the specificity criterion regardless of whether the underlying field is populated.

Step 5 — Verify attribution distinction. Where the change category includes both human-authored and AI-mediated possibilities, Q1's answer must distinguish them. A human exercising the override right and an LLM operating under an orchestration rule produce architecturally different changes — the former carries human authority directly, the latter indirectly through the rule the LLM executes under, per the AI-as-substrate-mediator commitment's attribution property. The test passes only when substrate provenance preserves this distinction; it fails when AI-mediated changes are recorded under a generic attribution that conflates them with human modifications.

Step 6 — Verify correctness. Each answer must be checkable against an independent reading of the change — typically by inspecting the substrate content the change produced and the orchestration rules in effect at the time. The test fails when answers are specific but wrong (a recorded rule reference that does not match the rule actually in effect; a recorded actor that does not match the change's structural fingerprint).

The procedure terminates when all sampled changes across all applicable categories have been interrogated against all four questions, with specificity, attribution distinction, and correctness verified for each.

4. Pass and fail conditions

The test passes if and only if, for every sampled change in every applicable category, all four questions are answerable from substrate provenance alone, with answers that are specific, correctly matched to the change, and properly attributed in the human-authoring/AI-mediation distinction.

The test fails under any of the following conditions:

- **Unanswerable.** One or more questions cannot be answered for one or more sampled changes; the relevant provenance is missing or unresolvable from substrate content alone.
- **Aggregated.** Q2's answer resolves only to a time window rather than a specific timestamp.
- **Generic.** Q3's answer resolves only to a class of authority rather than a specific orchestration rule, or Q4's answer resolves only to a generic mechanism name rather than a specific cell identifier.
- **Externally required.** An answer requires consulting external systems — vendor audit logs, LLM session histories, agent memory, external tool state — to be resolvable. Failures here include the canonical anti-pattern in which decisions are taken outside the substrate and then summarized into it, and the canonical anti-pattern in which writes happen to non-addressable surfaces.
- **Attribution-conflated.** Q1's answer does not distinguish between a direct human modification and an AI-mediated cell write under an orchestration rule, recording both under the same attribution token.

A failing test diagnoses which condition is responsible, so the failure is remediable rather than merely identified.

5. Anti-patterns the test specifically detects

The test detects a class of architectural failures that field-presence inspection alone does not. Five canonical anti-patterns illustrate the class.

Provenance-without-attribution-distinction. All six provenance fields are populated, but Q1's writer-attribution field carries the same token (e.g., "system," "service-account," "automation") for both human modifications and AI-mediated cell writes. The provenance-completeness test passes; this test fails on Q1, because the architectural distinction between direct human authority and AI-mediated authority is not preserved in the recorded answer.

Aggregated-time-window provenance. The timestamp field is populated, but the populated value is a session boundary or batch identifier rather than the specific moment of the change. Q2 is unanswerable at the specificity the commitment requires.

Generic-rule-references. The rule-reference field is populated, but the value is a class of rules ("data-update policy," "the editorial workflow") rather than a specific rule identifier resolvable to a specific orchestration rule. Q3 fails specificity.

Missing-mechanism-detail. The mechanism field is populated with a label that does not resolve to a specific cell identifier or named mechanism — "automated processing" or "system pipeline" rather than the specific cell that executed. Q4 fails specificity even though the field is non-empty.

Vendor-audit-logs-required. Substrate provenance is incomplete in a way the schema permits, and the missing pieces are reconstructable only by querying vendor audit dashboards, LLM session histories, or external tool state. The substrate-alone constraint is violated; the test fails on whichever question requires the external query, regardless of whether the answer is recoverable somewhere.

The test additionally detects the composition of these anti-patterns with the canonical retraceability-violating patterns named elsewhere in the derivation series — non-addressable writes, in which writes occur to surfaces the substrate cannot index for the four questions; and external-tool-state-authoritative configurations, in which the source of truth for a question's answer lies outside the substrate. Both fail this test on whichever question the missing or external content was supposed to support.

6. Integration with deployment verification

The test integrates with deployment verification at five touchpoints.

Initial deployment validation. Before substrate activation, the test runs against any seed substrate content the deployment carries to confirm that question answerability holds from the start. Substrates that fail this validation are not promoted to active use.

Audit-readiness verification. Before scheduled audits, internal or external, the test runs to confirm the substrate's provenance content supports the four-question interrogation auditors will perform. A failed test produces a remediation backlog with anti-pattern diagnoses naming which provenance content needs strengthening before audit exposure.

Compliance review. When provenance content is reviewed under a compliance framework, the test runs as the operational check that the substrate's traceability commitment is in fact instantiated, distinct from documenting that the commitment is intended.

Composition partner verification. When CKS substrates compose, four-question answerability must hold across composition boundaries. A change made in one substrate must remain answerable when read through the composed view; otherwise composition silently degrades retraceability. The test runs in the composed configuration to verify this property.

Vendor migration verification. When substrate infrastructure migrates between vendors, the test runs in the post-migration configuration to verify that question answerability has been preserved through migration. This is the moment most prone to silent regression — vendor-specific provenance representations may not transfer cleanly between platforms — and the test catches such regressions before they accumulate.

7. Limits of the test

Stating the test's limits precisely is what keeps it useful as a verification procedure rather than overreaching as a quality measure.

The test does not verify that the six provenance fields are present. That is a different test — the provenance-completeness sibling test — that verifies the architectural mechanism. The

four-accountability-questions test verifies the operational outcome, given fields present.

The test does not verify other aspects of path retraceability beyond the four questions. Path retraceability also includes the substrate-only-paths property, the antecedent-references property, and the conflict-relationship property — each with its own derivation note and operational considerations. This test is scoped to the four questions specifically.

The test does not verify whether the recorded changes were appropriate. It is silent on whether a given override should have been exercised, whether a given rule should have been authored, whether a given AI-mediated change should have been authorized. Those are governance questions the architecture supports asking but does not itself answer.

The test does not verify compliance with any specific compliance framework. SOX, HIPAA, GDPR, and similar frameworks may impose their own provenance requirements with their own answerability standards; the test verifies the CKS commitment, not those frameworks'. A substrate that passes this test may still fail a framework-specific audit because of framework-specific requirements; a substrate that passes a framework-specific audit may still fail this test because the architectural commitment is stricter or differently scoped.

8. The test in one sentence

A CKS deployment passes the four-accountability-questions test if and only if, for every substrate change across every applicable change category, the four questions — who made the change, when it was made, under what authority or rule, and through what mechanism — are answerable from substrate provenance alone, with answers that are specific, correct, and properly attributed in the human-authoring/AI-mediation distinction.

9. Why naming this test as standalone matters

Question answerability is the operational outcome path retraceability is designed to support. Naming the test that verifies that outcome as a standalone procedure has three consequences.

It separates architectural mechanism from operational outcome. The provenance-completeness sibling test verifies the mechanism; this test verifies the outcome. A deployment can pass the former and fail the latter, and the failure mode names something the former does not catch — provenance fields populated but not informative, populated but not specific, populated but not attribution-distinguishing. Without the standalone test, that failure mode is undetectable until an actual audit surfaces it under conditions where remediation is more costly.

It closes the path retraceability operational verification pair. Together with the provenance-completeness sibling test, this test verifies the path retraceability commitment in two halves — fields present and questions answerable. A deployment that passes both has operationally verified path retraceability; a deployment that passes only one has verified only the half it passed. The pair structure is what makes path retraceability operationally testable rather than only architecturally specifiable.

It positions the next operational test in the derivation series. The substrate-as-source-of-truth commitment names five categories of authoritative state the substrate must carry; the next operational test formalizes the procedure for verifying those five categories are all present and authoritative. The progression — from this path retraceability test pair through source-of-truth, tool-agnosticism, cost, conflict, composition, pattern-mapping, and reproducibility tests — covers the operational verification surface of the CKS pattern's architectural commitments, each formalized as standalone prior art under the same convention.

Subsequent work that adopts the CKS pattern, instantiates it, or composes it with adjacent patterns should treat the four-accountability-questions test as the verification procedure that closes path retraceability's outcome side, paired with the provenance-completeness test that closes its mechanism side. Subsequent work that uses a different verification procedure for the same commitment is using a different procedure, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Four-Accountability-Questions Test as Standalone Procedure Specification: A Verification Procedure for Path Retracement in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.